# AutoISF3.2.0 – Quick User Guide

**Disclaimer:** *I am not a medically trained person and developed these methods purely based on trial and numerical experimentation. For example, I had no mathematical model of the reaction kinetics of the free fatty acids which are one of the reasons for temporary insulin resistance. If you want to try some of these features yourself, do it at your own risk and your own liability.*

## Preface

The purpose of this document is to describe the features of AutoISF and their settings. Hints for specific tuning will be given in several "How-to-…" guides which are under development or planned. These tuning hints' effects differ from person to person and depend on your planned usage scenario of AutoISF as just testing it, fine tuning AAPS, low carb diet, extensive endurance exercise, or full closed loop, i.e. no announcing of carbs and no manually triggered boluses.

The Quick Guides for earlier versions of AutoISF were merged into this single document which also introduces the new features. AAPS 3.3.x versions contain a subset of AutoISF already in a separate plugin., i.e. it is not integrated in the SMB plugin like before. Only this release contains all the features.

The focus of this release was on combining AutoISF with recent AAPS3.4.0.0 master as well as adding visual information about key AutoISF interim values:

- The fitted parabola can now be shown as overlay in the main BG graph.
- The history of individual auto_ISF factors can be shown in the smaller graphs.
- The history of iob_TH can be shown in a smaller graph
- The AAPSClient app can now show the last Script Debug from the master phone.

In versions AutoISF 3.1.0 only, a bug was present related to calculating the correlation of the parabola fit. This correlation was mostly too high, i.e. pretended too good a fit. If at all, with this corrected correlation less insulin might be given. Therefore the correction is not critical. Please note that versions before AutoISF 3.1.0 or the AUTOISF plugin in master AAPS do not contain that bug.

In version AAPS3.3.3.a+aisf3.1.0 only, a bug was present related to the carb calculator and the carb absorption. Both should use the 24 hour average variable_sens from AutoISF but used plain profile ISF instead. If you enter carbs in the calculator you may see changes in carb decay times. Please note that versions before AAPS3.3.3.a+aisf3.1.0 or the AUTOISF plugin in master AAPS do not contain that bug.

**A word of caution**: This version of AutoISF adds a table holding the graphical data to the AAPS database. Therefore it is not compatible with vanilla AAPS or previous releases of AutoISF although simple updating the install may work. Therefore, and as always with a new versions, first export and backup your previous up-to-date settings.

# Introduction

AutoISF is meant for the advanced user who has a deep understanding of AAPS and who has tuned the system to achieve a TIR of about 90% or better. If such a user is ambitious and wants to improve further, the methods contained in AutoISF can help achieve that goal.

This document describes how and when ISF is adapted automatically and provides short descriptions of the weighting factors to tune it to your individual needs. Detailed guides covering special situations and examples will follow later as separate documents including results of using AutoISF in full closed loop mode, i.e. without carb or insulin entry by the user. Furthermore, it describes alternative methods on how to scale, activate or deactivate SMBs. Finally there are methods for handling activity levels ranging from mild up to intensive.

The adaptation of ISF is based on special glucose behaviour and results in an adaptation factor just like the Autosense factor. However, in AutoISF only ISF is adapted and no other settings are modified. The scenarios are analysed typically cover the last 10-30 minutes and therefore AutoISF reacts much faster to blood glucose variations. Often Autosense would drive me into hypos because of its delayed reaction even after blood glucose levels had come back on target. As a result, I disabled Autosense although mathematically both can coexist.

AutoISF is part of oref1 in OpenAPS SMB plugin and cannot coexist with the recent DynamicISF which therefore is included in its own plugin as an alternative to OpenAPS AMA and OpenAPS SMB.

Please note that the rapid adaptations of ISF by AutoISF render the use of Autosense and Autotune useless because those would draw conclusions based on constant ISF or only slow changes and therefore false assumptions. If you want to use Autotune then disable those AutoISF features which adapt ISF for those Autotune testing periods.

There are 4 different effects in glucose behaviour that AutoISF checks and reacts to:

1. **acce_ISF** is a factor derived from acceleration of glucose levels
2. **bg_ISF** is a factor derived from the deviation of glucose from target
3. **pp_ISF** is a factor derived from the delta in glucose levels
4. **dura_ISF** is a factor derived from glucose being stuck at high levels

Finally these factors are compared among each other and against Autosense. Normally the strongest of them will be used with some exceptions as detailed further below. These factors work the same way as the Autosense factor works, i.e. the profile sensitivity ISF is divided by the factor to deliver a final sensitivity ISF.

In the AUTOISF tab, section Script debug, you can always see which values were assigned to the 4 factors shown above during the last loop execution. It also lists explanations in case the factor had to be modified or why it cannot be used. Some interim values like *dura_ISF_average* are listed, too. All the resulting data can be seen in the AUTOISF tab sections Glucose status, Profile and Script debug, respectively.

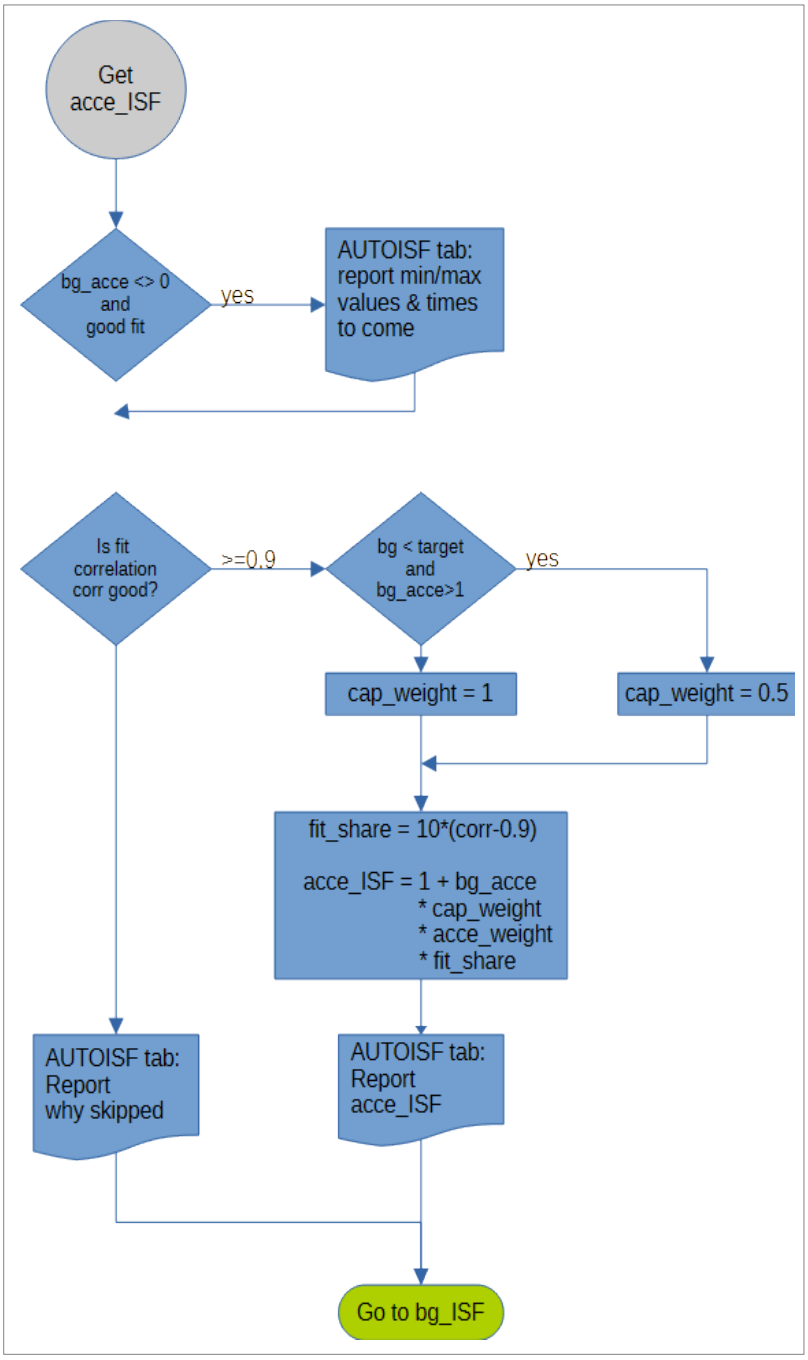
Again in analogy to Autosense there are upper (*autoISF_max*) and lower (*autoISF_min*) limits for how far ISF can be modified in total.

In analogy to enabling SMB, there is a setting *enable_autoISF* which determines whether all of the 4 ISF adaptations of AutoISF listed above are enabled or none at all.

The settings specific to AutoISF are collected in its own menu found at the bottom of the AUTOISF preferences menu. A screenshot is shown as attachment at the end of this document. Another trick for finding them is to use the search icon at the top of the Preferences page which searches for all settings containing the string you enter in the search field.

Status: 14/02/26 @ 15:37:20 Back to top

# acce_ISF determination and its impact




This is the latest contribution to AutoISF and has been in use since late December 2021.

For things like glucose or delta to change, there first must be an acceleration. Therefore, acceleration recognises such changes earlier and is used by AutoISF to take pre-emptive action.

Glucose and delta, its 1st derivative, play a significant role in AAPS in determining the insulin required. Acceleration, its 2nd derivative, was not included so far. One reason might be that it is harder to extract from the glucose history considering that delta needed to be averaged already in order to provide a reliable signal. In AutoISF a best fit algorithm is used to determine the parabola which best matches the glucose data. That implicitly includes a smoothing effect.

Once the formula for the parabola is known, it is then very easy to determine the acceleration. Sometimes the fit has bad correlation, i.e. the glucose readings deviate too much from the fit. In such cases there is no contribution from acceleration and acce_ISF = 1.

Otherwise acce_ISF is calculated by

acce_ISF = 1 + acce_weight * fit_share * cap_weight * acceleration

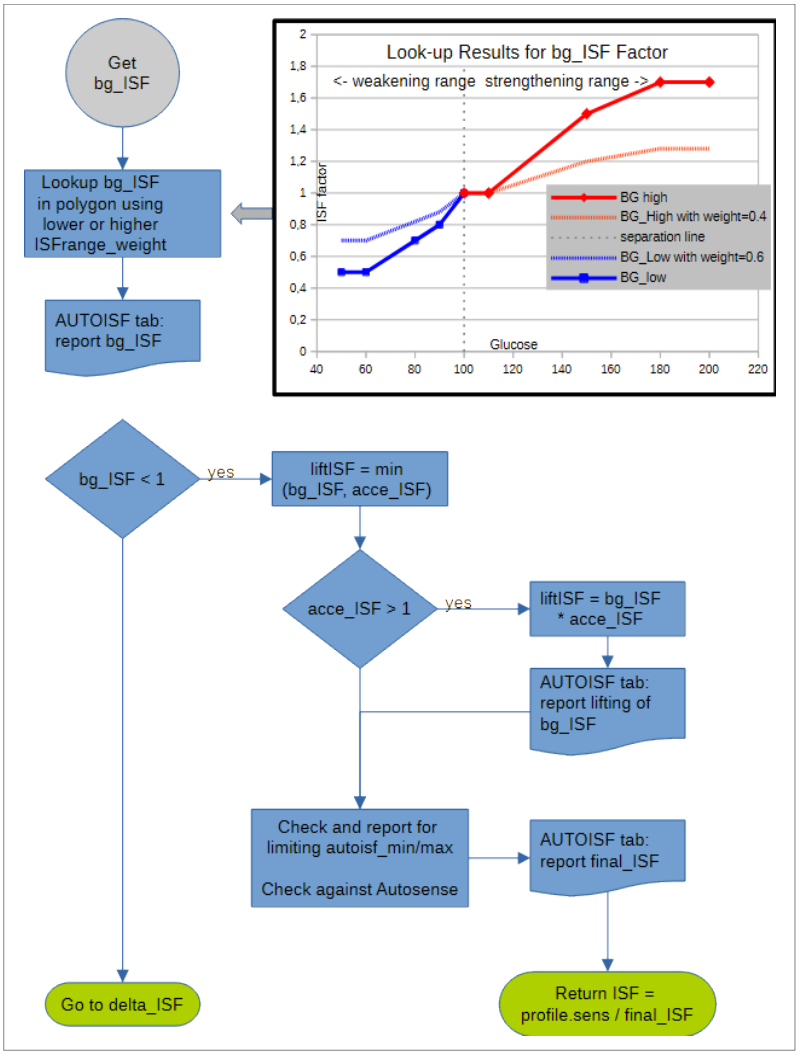
where fit_share a measure of fit quality, i.e. 0% if unacceptable up to 100% if perfect;
cap_weight is equal to 0.5 below target and 1.0 otherwise;
acce_weight is equal to bgAccel_ISF_weight for positive acceleration (in general)
or is equal to bgBrake_ISF_weight for negative acceleration (in general)

Experiences say that the weight while decelerating should be 30-40% of the acceleration weight in order to reduce glucose oscillations. Quite often the acce_ISF contribution plays the dominant role inside AutoISF and is therefore very important and delicate. Weights for acce_ISF of 0 disable this contribution. Start small with weights like 0.02 and observe the results before increasing them. Keep in mind that negative acceleration will start to happen while glucose is apparently still rising but the slope reduces. Here, acce_ISF will be <1, i.e. sensitivity grows and less insulin than normal will be required even before the glucose peak is reached.

# bg_ISF determination and its impact

There are indicators that higher glucose needs a stronger ISF. This was evident from all the successful AAPS users defining automation rules which strengthen the profile at higher glucose levels. The drawback is that there are sudden jumps in ISF at switch points and no adaptations in between.

In AutoISF a polygon is provided that defines a relationship between glucose and ISF values and interpolates between them. This is currently hard coded but the user can apply weights to easily strengthen or weaken it in order to fit personal needs. In principle the polygon itself can be edited and the apk rebuilt if a different shape is required. Developing a GUI for that purpose was considered very tedious, especially before knowing whether the results warrant the effort.




There are two weighting factors depending on whether glucose is below or above target:

| | |
|---|---|
| *lower_ISFrange_weight* | Used below target, weakens ISF the more the higher this weight is; 0 disables this contribution, i.e. ISF is constant in the whole range below target.<br>This weight is less critical as the loop is probably running at TBR=0 anyway and you can start around 0.2. |
| *higher_ISFrange_weight* | Used above target, strengthens ISF the more the higher this weight is<br>0 disables this contribution, i.e. ISF is constant in the whole range above target.<br>Start with a weight of 0.2 and observe the reactions and check the AUTOISF tab before you increase it with care. |

The result is:

bg_ISF = 1 + *xxx*_ISFrange_weight * glucose_polygon_Lookup

There is a special case possible, namely below target i.e. when bg_ISF<1. ISF will be weakened and there is no point in checking the remaining effects. Only with positive acceleration the weakening will be less pronounced as that is a sign of rising glucose to come soon.

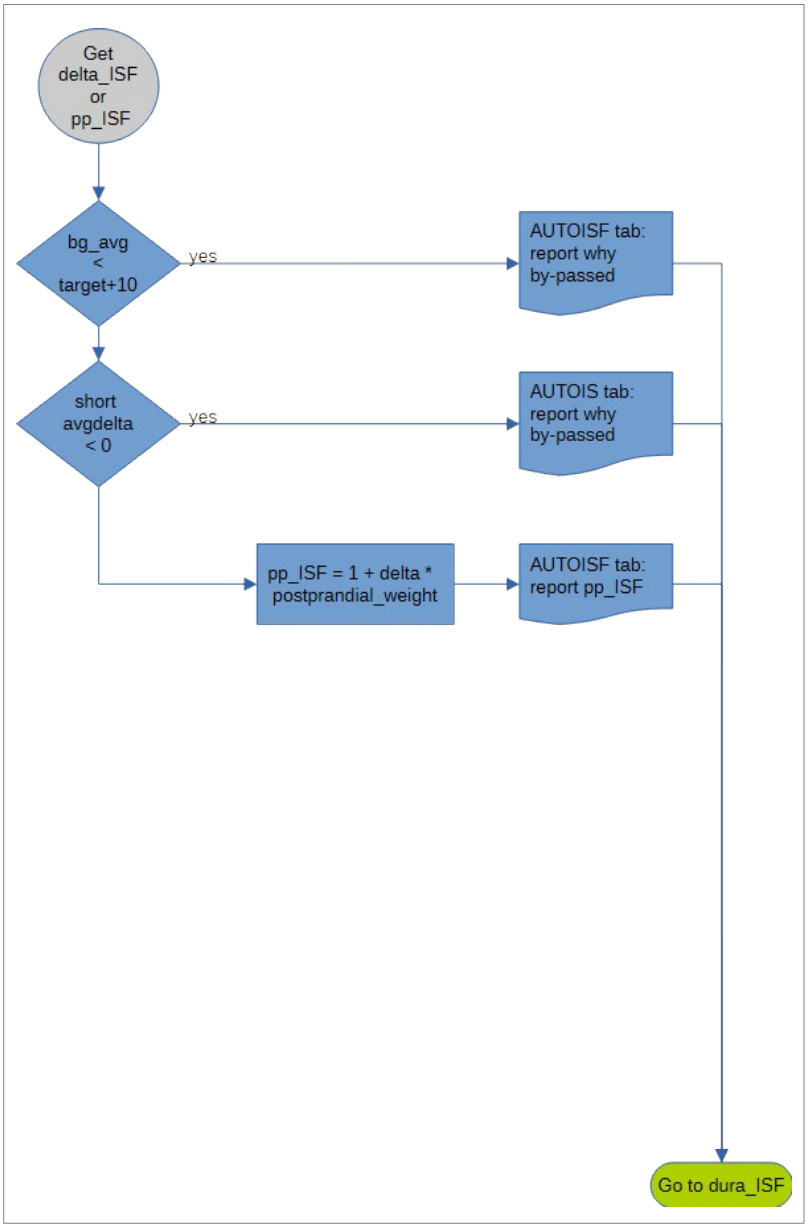
# pp_ISF determination and its impact

AutoISF can adapt ISF based on glucose delta. It was introduced to help users with gastroparesis. It is also useful for users in pure UAM mode because in their case no meal start can be detected. Given a positive *short_avgdelta* and glucose being above target+10 the result is:

pp_ISF = 1 +
delta * pp_ISF_weight.

As a starting value for *pp_ISF_weight* use 0.005. Observe the reactions and check the AUTOISF tab before you increase it with care.

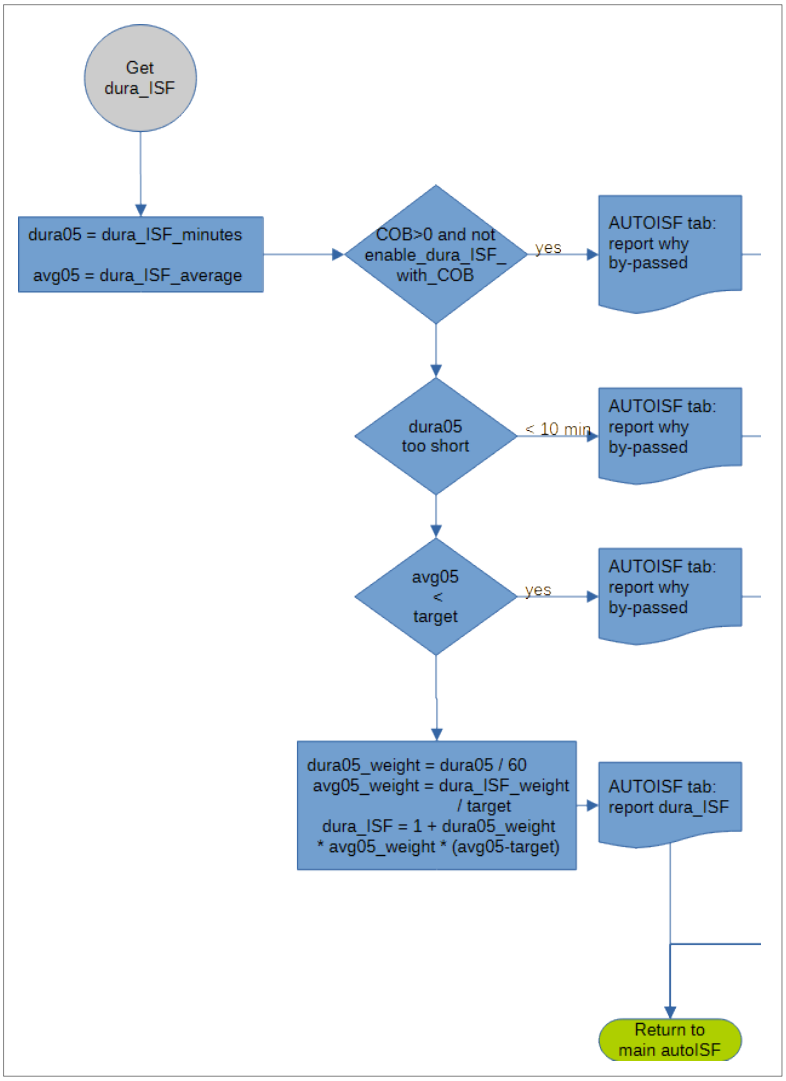
A weight of 0 disables this contribution.



# dura_ISF determination and its impact

This is the original effect of AutoISF in action since August 2020. Because AutoISF is now a toolbox of several effects this original effect was renamed *dura_…* It addresses situations when

- glucose is varying within a +/- 5% interval only;
- the average glucose (*dura_ISF_average)* within that interval is above target;
- this situation lasted at least for the last 10 minutes (*dura_ISF_minutes)*.




This is a classical insulin resistance and is typically caused by free fatty acids which grab available insulin before glucose can. Quite often users get impatient in such a situation and administer one or even more “rage boluses”. Again and again that leads to hypos. The dura_ISF approach avoids this if it is carefully tuned.

The strengthening of ISF is stronger the longer the situation lasts and the higher the average glucose is above target:

$$\text{dura\_ISF} = 1 + \frac{\text{avg05-target\_bg}}{\text{target\_bg}} * \frac{\text{dura05}}{60} * \text{dura\_ISF\_weight}$$

where

avg05 = dura_ISF_average
dura05 = dura_ISF_minutes

The user can apply his personal weighting by using *dura_ISF_weight*. Start cautiously with a value of 0.2 and be very careful when you approach 1.5 or even higher. By using 0 this effect is disabled.

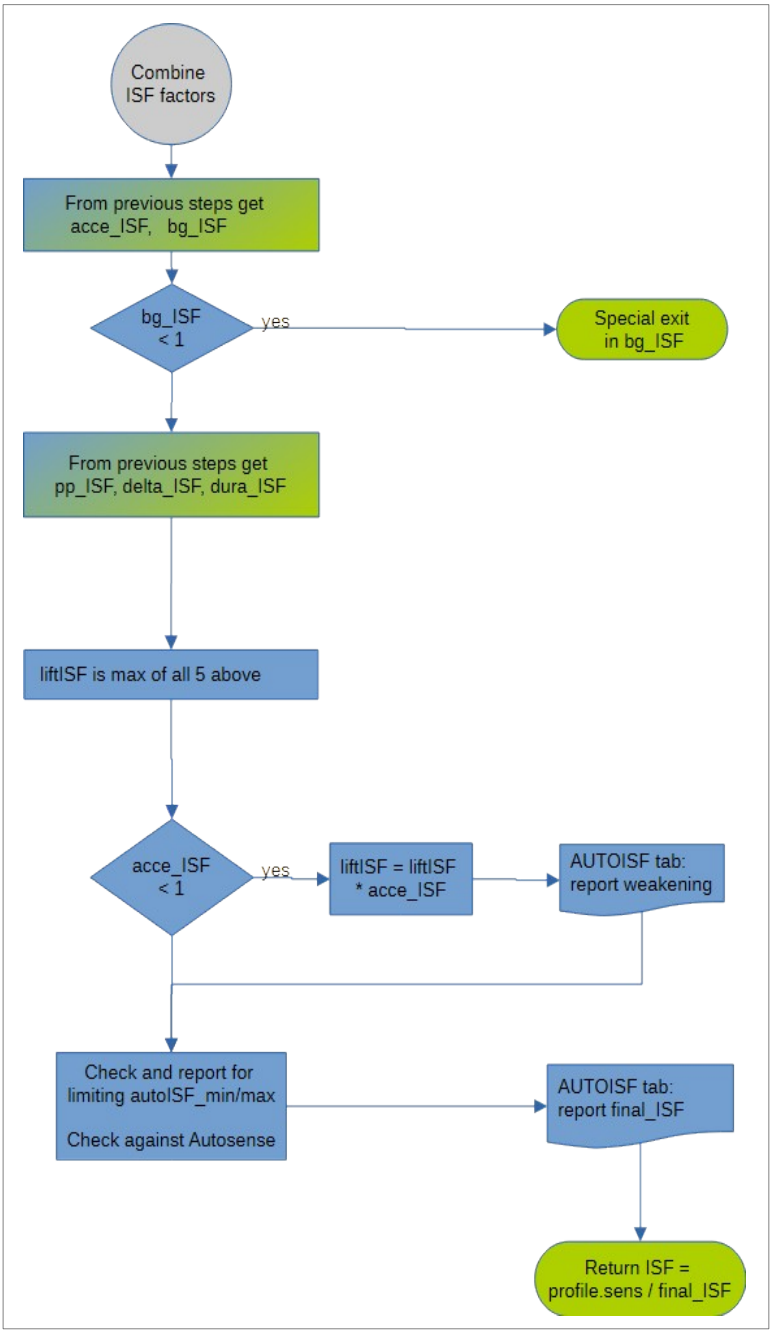
# The combined result from the above factors

Now that the factors for all 4 effects are known, how to deduce an end result? The normal case is to pick the strongest factor as the one and only factor to be applied. Here autosense is also part of the game. But how about the exceptions, i.e. when different factors pull in different directions? In order of precedence they are:

- bg_ISF < 1, i.e. glucose is below target
  If acce_ISF>1, i.e. glucose is accelerating although below target, both factors get multiplied as a trade-off between them. Then the weaker of bg_ISF and Autosens is used as the final sensitivity ISF.

- acce_ISF < 1, i.e. glucose is decelerating while other effects want to strengthen ISF.
  In this case the strongest of the remaining, positive factors will be multiplied by acce_ISF to reach a compromise. This overall factor will be compared with autosense and the stronger of the two will be used in calculating the final sensitivity ISF.

In all of the above the AutoISF limits for maximum and minimum changes will also be applied.




In order to see how the strengths of the individual contributions compare against each other, you can use the new options for plotting those in the smaller graphs. This even works for past times by going to the history browser tab.

Status: 14/02/26 @ 15:37:20

Back to top

# Exercise mode

### Why can’t I use Exercise mode in standard AAPS but with AutoISF?

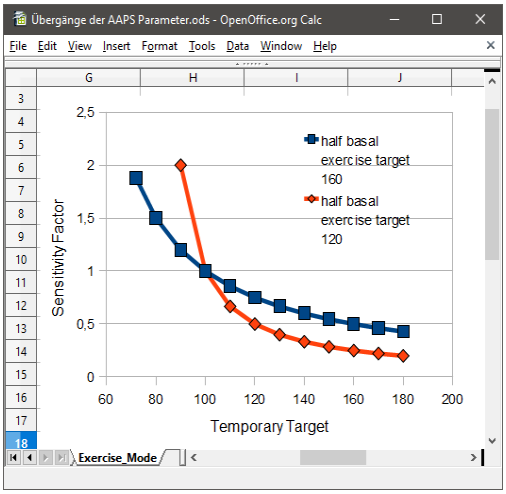
Exercise mode was disabled in AAPS years ago because there is some risk. In AutoISF it is enabled like it is in OpenAPS, too. However, because it is a powerful tool, it should be used with caution. Scott Leibrandt once described the risks as follows:

*"It was a rather rare case where the sensitivity could be too high. It was really a very rare case, you had to meet all of the following 5 conditions.*
*(Number 5 is a safety setting! You have to have played around with the default settings).*

1. *Oref1 sensitivity enabled*
2. *High temptarget raises sensitivity enabled.*
3. *A site change or profile change has been logged within the last few minutes.*
4. *A high temptarget is enabled.*
5. *An unreasonable autosens threshold has been set."*

The toggle on the overview screen was added to quickly deactivate (top) or activate (bottom) the exercise mode. Otherwise the settings can also be adapted individually in the AutoISF menu. The relevant settings for exercise mode are:


- *high_temptarget_raises_sensitivity* (synonym for exercise mode); defaults to false.
  When set to true, temp targets > 100 raise sensitivity (lower sensitivity ratio / higher ISF numbers) and reduce basal. The higher your temp target is above 100 the more sensitive the ratios will be, e.g. temp target of 120 results in sensitivity ratio of 0.75, while 140 results in 0.6 (with default *half_basal_exercise_target* of 160). Basal will be reduced accordingly. See also *half_basal_exercise_target.*




- *half_basal_exercise_target*; defaults to 160.
  This means means when temp target is 160 mg/dl and *high_temptarget_raises_sensitivity* =true then basal will run at 50% (for TT=120 at 75%; for TT=140 at 60%). This base exercise target number can be adjusted in the AutoISF menu to give you more control over your exercise modes. In addition to basal, it also influences sensitivity. See also *high_temptarget_raises_sensitivity.*
- *low_temptarget_lowers_sensitivity* (synonym for resistance mode); defaults to false.
  When set to true it can lower sensitivity (higher sensitivity ratio/lower ISF numbers) for temp targets < 100. The lower your temp target is below 100 the less sensitive the ratios will be, e.g. temp target of 95 results in sensitivity ratio of 1.09, while 85 results in 1.33 (with default *half_basal_exercise_target* of 160). For safety reasons the maximum effect is limited to 1.5 and to *autosens_max*, whichever is less powerful. Here too, the basal is also adjusted and increased accordingly.

**Please note**

The sensitivity ratio (ISF) modified by exercise or resistance mode is the basis for further sensitivity modifications by AutoISF. This is an exception to the basic rule that only the stronger of ISF modifier prevails.

## Activity Monitor

This is a milder method of adapting sensitivity compared to the exercise mode. Examples of when it appears to work well is for vacuum cleaning, a short walk to the cinema or a short bicycle ride for shopping. The base information comes from the phones acceleration sensor and its inbuilt step counter. There is a new switch in the AutoISF menu to enable the activity monitor. By default it remains inactive. The steps counted can be checked in the AUTOISF tab towards the end of the profile section. The steps are evaluated for various time segments during the last hour and lead to these 5 classifications:

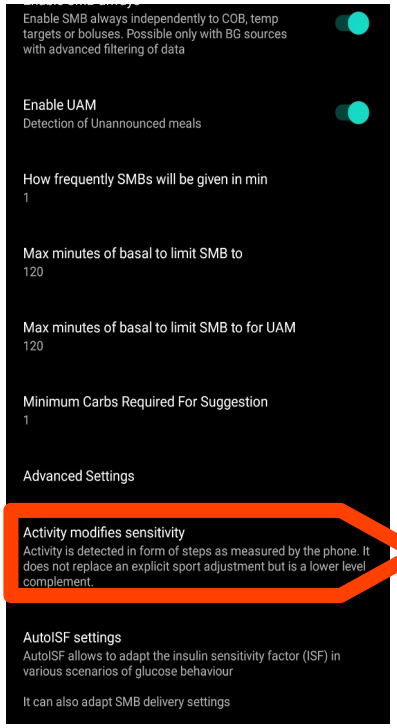
| Classification | Description | Max 1.5 | Default 1.0 | Example 0.6 | Min 0.0 |
|---|---|---|---|---|---|
| activity | step count fairly above neutral | 0.55 | 0.70 | 0.82 | 1.00 |
| partial activity | step count somewhat above neutral | 0.775 | 0.85 | 0.94 | 1.00 |
| neutral | neutral step count, loop tuned accordingly | 1.00 | 1.00 | 1.00 | 1.00 |
| partial inactivity | step count somewhat below neutral | 1.15 | 1.10 | 1.06 | 1.00 |
| inactivity | step count fairly below neutral | 1.30 | 1.20 | 1.12 | 1.00 |

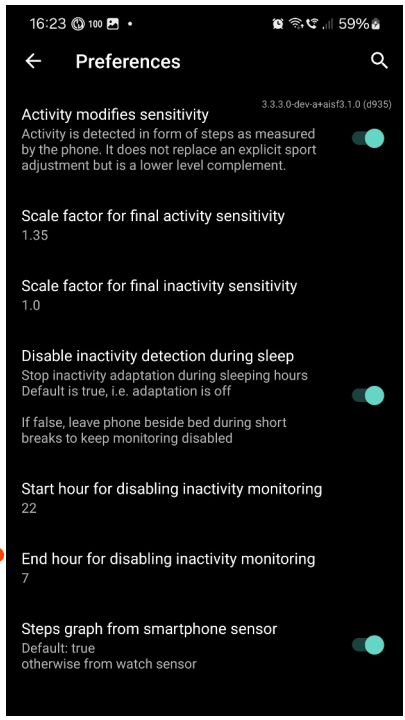
The columns to the right show resulting sensitivity change factors for various scale factors. These scale factors modify the default and can be set in the Activity Monitor preferences to adapt the impact to your personal needs. Right at the start of the AUTOISF tab debug section the activity monitor lists the current status and assessment of activity level.

Besides the sensitivity, the basal rate is also adapted. The unexpected but welcomed side effect of the 1 hour time window holding step counts is the gradual phasing out after the end of activity from 30% via 15% to eventually 0% impact. This also works at the end of exercise mode because activity monitor takes over once the exercise TT ends.

The activity monitor is disabled or limited in either of the following situations:

- no inactivity detection during the first hour of (re-)starting AAPS
- deactivated in settings
- a TT is active
- the phone was not carried during the last 15 minutes




This last exception leads to the question what to do during sleeping hours. With the phone being at rest how can you avoid ending up in inactivity mode if sleep is interrupted and you quickly check the phone or go to the bathroom?

Option 1: just leave the phone next to the bed and don't touch it; trust the loop or check status on the watch.

Option 2: enable and define your personal window of sleeping hours to disable the inactivity detection.

Option 3: see pilot application of state automation

Once you get up for good you definitely start the day in inactivity mode. This is a welcome state to fight the increased resistance during the dawn phenomenon.

Status: 14/02/26 @ 15:37:20

Back to top

## Internal automation for iobTH

The variable *iob_threshold_percent* holds a percentage of the max_iob which is used as the threshold to disable SMB. Inside the AutoISF code it offers more flexibility than can be achieved with regular AAPS automations used before. The result of any sensitivity change defined by the user is a modulated value labelled internally as effective iobTH.

The new capabilities are:

- *iob_threshold_percent* gets modulated while the pump profile is set to a percentage. The idea is that with changed sensitivity the threshold should change accordingly. So internally an effective iobTH is used. If for example the profile is raised to 120% because of an infection then the effective iobTH is 120% of *iob_threshold_percent*. This relieves the user from having to adapt the automation rules for those periods and having to remember setting them back once the profile is reset.
- *iob_threshold_percent* gets modulated while exercise mode is active which implicitly changes sensitivity. The reasoning and rules are the same as in the preceding situation. This modulation is combined with the above profile based modulation.
- *iob_threshold_percent* gets modulated while the activity monitor changes sensitivity. The reasoning and rules are the same as in the preceding situations. This modulation is combined with the above profile based modulation.
- A very special modification happens during the initial rise after carbs intake. After the first few SMBs the iob threshold may eventually be surpassed. Often this initial overshoot was far too much due to limited capabilities using automations and led to hypo later. The code will limit this overshoot or tolerance to 130% of the effective iobTH. During the next loop the iob will most probably still be above that threshold and therefore SMBs stay disabled until IOB drops below the effective threshold.

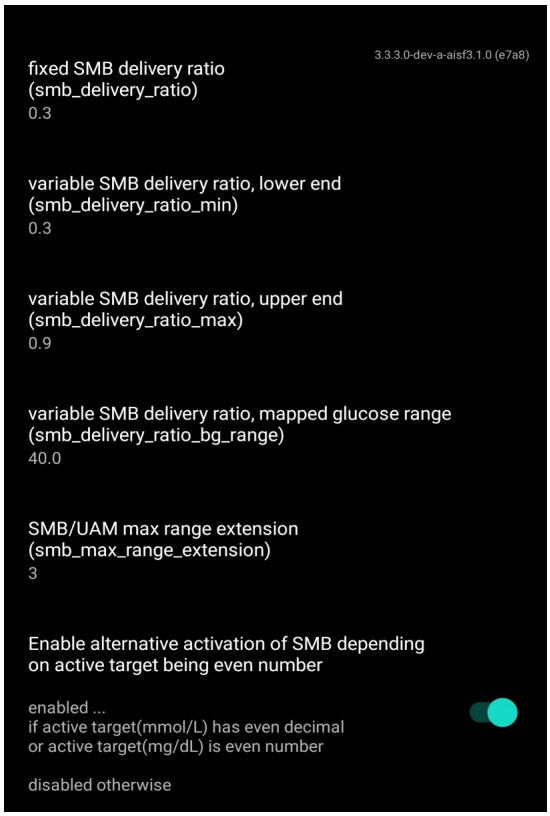
You may want to consider raising your “classical” iob threshold values used by automations because the new 130% overrun will end up in lower values on average. But then the 130% is not reached every time and raising them by 10%-20% may be a good initial compromise.

The value *iob_threshold_percent* is accessible for manual editing or inspection in the AutoISF Menu system. Its current value is shown in the SMB tab right at the start of the AutoISF echo together with its modulation.

To get an impression of how iobTH compares to IOB or COB you can show their time histories superimposed in a single chart. There you also see the modulation of iobTH as caused by underlying sensitivity adaptions like exercise mode or changes in profile percentage.

If the value is set to 100%, the function will be deactivated

# Adapting SMB delivery

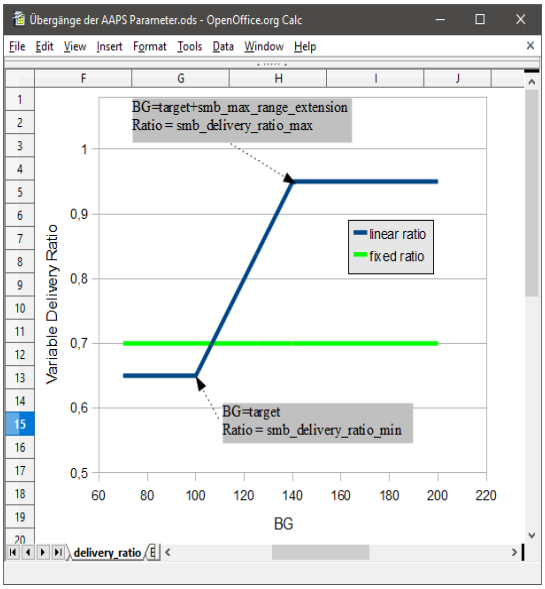



After reading all the methods to strengthen ISF you may wonder whether your maxBolus will always allow as much SMB as requested by the loop with such stronger ISF. This is especially important for pure UAM mode, where you want a few but strong SMBs as soon as meal absorption is detected by acce_ISF in order to catch up with pre-bolus and meal bolus in standard use of AAPS. Several extensions in AutoISF can be used to get there:

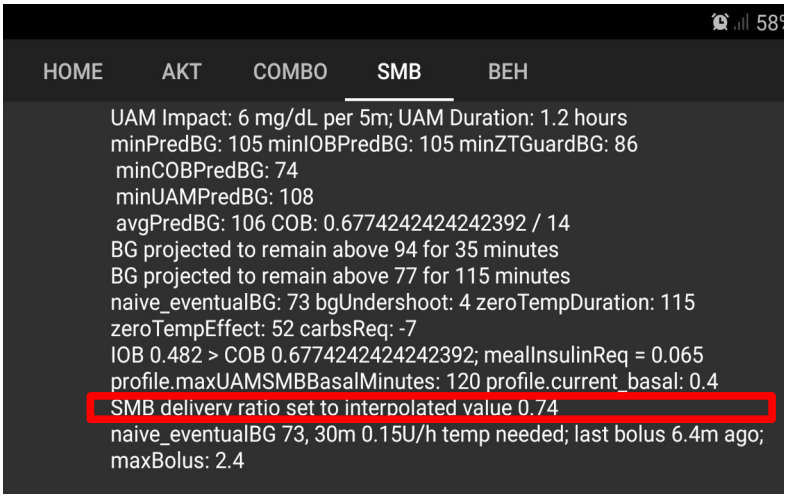
- In AAPS *smb_delivery_ratio* is normally hard coded as 0.5 of the insulin requested. This is a safety feature for master/follower setups in case both phones trigger an SMB in the same situation. If this does not apply in your case you may increase this setting to a value above 0.5 and up to even 1.0 if you are very courageous. Best is to leave some margin like the max delivery setting in the bolus calculator.

- Alternatively to a higher but fixed ratio you can use a linearly rising ratio, starting cautiously with *smb_delivery_ratio_min* at *target_bg* and rising to a more ambitious *smb_delivery_ratio_max* at *target_bg+smb_delivery_ratio_bg_range*. If *smb_delivery_ratio_bg_range=0* then this linear rise is disabled and the above *smb_delivery_ratio* is used instead.

- When using Libre see also the Libre section for using ratios below 0.5 if in 1 minute mode for CGM and SMB interval.



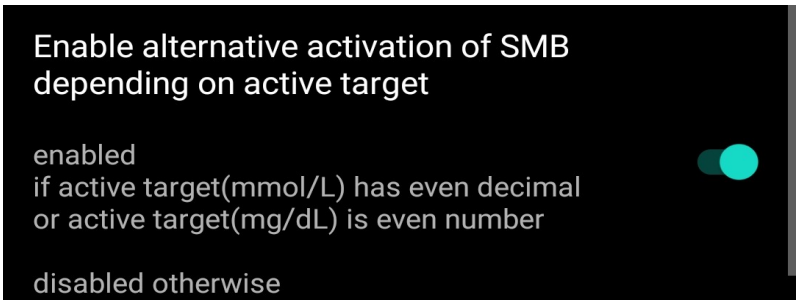

- *smb_max_range_extension* is a factor that effectively multiplies the current *maxSMBBasalMinutes* and *maxUAMSMBBasalMinutes* beyond the 120 minute limit.




Status: 14/02/26 @ 15:37:20

Back to top

# Alternatively enable SMB depending on target value type




This feature works independently of other AutoISF settings. By clever selection of targets, you now have additional options for enabling or disabling SMB.  This works across the whole allowed range of 72-180mg/dl. The decision is based on whether the target value is an even or odd number. For systems using mmol/l, it is slightly adapted as the number after the decimal point is taken into account : 5.2 is considered even, 5.1 is considered odd.

- As long as the current target is an even number, then SMB is always enabled and a message like "*SMB enabled; current target 78 is even number*" or "*SMB enabled; current target 5.2 has even decimal*" shows up in the AUTOISF tab. This is useful for *Eating Soon* and compatible with its standard assignment of 72. Also, you could for instance select 120 while sick in bed and still get SMBs, regardless of all the other SMB settings.
  The only exceptions are those situations where SMBs are disabled for reasons other than direct SMB preference settings, e.g. a hypo looming in the predictions.
- As long as the current target is an odd number, the SMB is always disabled and a message like "*SMB disabled; current target 81 is odd number*" or "*SMB disabled; current target 5.1 has odd decimal*" shows up in the AUTOISF tab. This is useful for calm periods or overnight with smoother curves by selecting TT=81 or 83 which worked very well for me. You can also use it overnight to avoid overreactions against compression lows.
- Please note that, in order to be used with regular profile targets, the lower and upper targets in the profile need to be identical.
- If none of the above applies or this features is disabled then the normal AAPS rules and messages will apply.

You need to be a careful because of **old habits** when defining a TT. *Eating Soon* at TT=72 behaves as before but *Hypo Target* at TT=120 would enable SMB which is probably not what you want in that situation. So better go to settings for *Default Temp-Targets* and change the *hypo target* to 121. You should also check the defaults for *Activity Target* and make sure that it fits your preferred SMB option during exercise.

With **automations** this offers a wide range of TTs and options without the need to adhere to the watershed at 100mg/dl. As an example take the situation where your IOB gets too high but the carbs are still coming in you can set a TT=73 by automation which gives the strongest possible TBR action but no SMB. **You absolutely must check all existing rules that set a TT whether they must be adapted.**

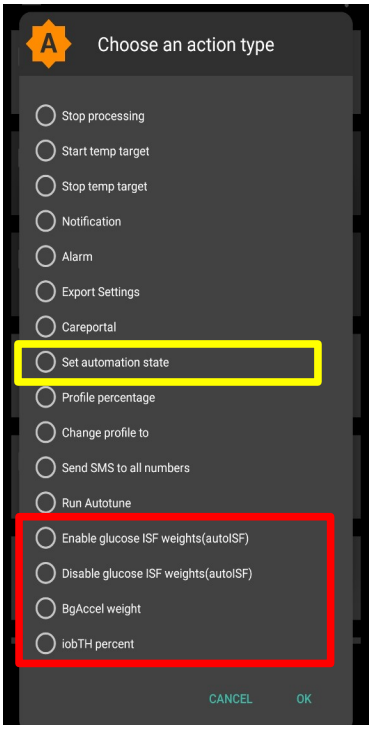
As you can see these new options are powerful but need careful preparation. Therefore the SMB delivery settings menu was extended as shown above by the switch *enableSMB_EvenOn_OddOff_always* to consciously enable this behaviour. Otherwise users not aware of this might get caught out.

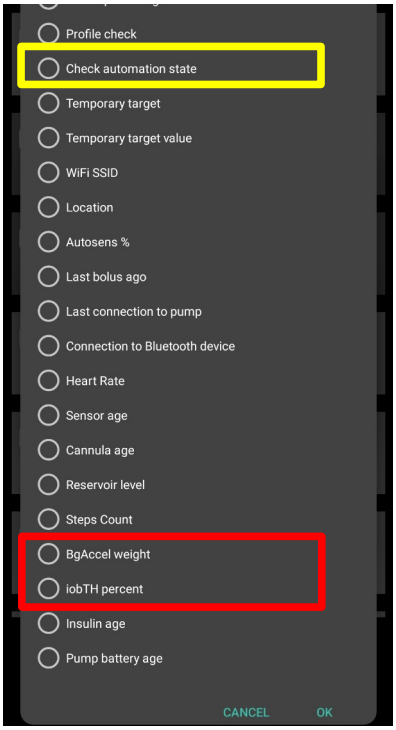
## Automations specific to the 4 glucose ISF weights

Following requests from several users some automations were developed and added with relevant AutoISF usage in mind.

### *Enable glucose ISF weights – Disable glucose ISF weights*

This will set the variable *enable_autoISF* accordingly. It only affects the 4 weights related to glucose behaviour and none of the other features like those influencing SMB.




### *Set BgAccel weight – Set iobTH percent*

This will set the two variables. For details of the new variable *iob_threshold_percentage* see the separate topic in this document.

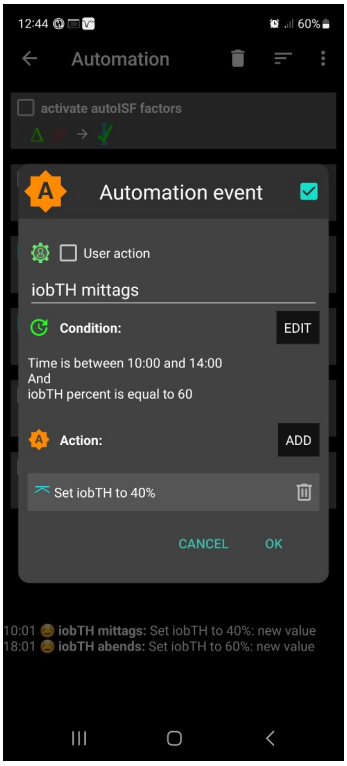
### *Trigger on BgAccel weight - Trigger on iobTH percent*

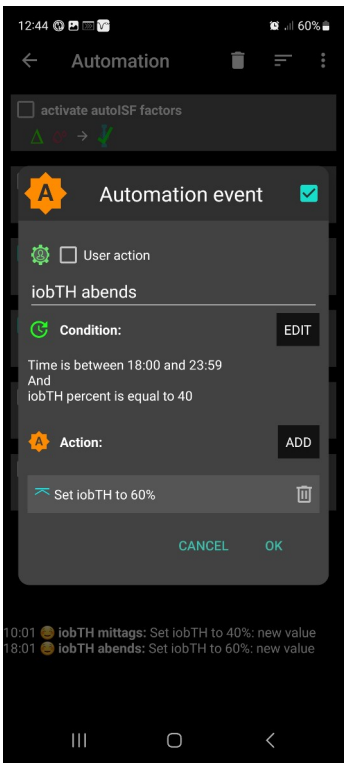
You can create your usual conditions like “equals” or “less than” against the current value of any of those two variables.

Here is an example set of automations to alternate between two values of iobTH:

Suppose I want to use two different values of *iob_threshold_percent* during a normal day. It is 40% for lunch time and 60% for dinner time. I have these two rules to switch by time of day and only if the current value equals the value from the earlier shift. Any other value is treated as a manual override for special occasions until I manually set it to its regular value. The time windows for switching are long enough to catch an opportunity to be processed and do not need to be actioned half a day each.

The often mentioned extra load on the phone battery is acceptable to me. With the automation plugin disabled completely a full charge lasts nearly 40 hours, with these two active it still lasts nearly 30 hours.



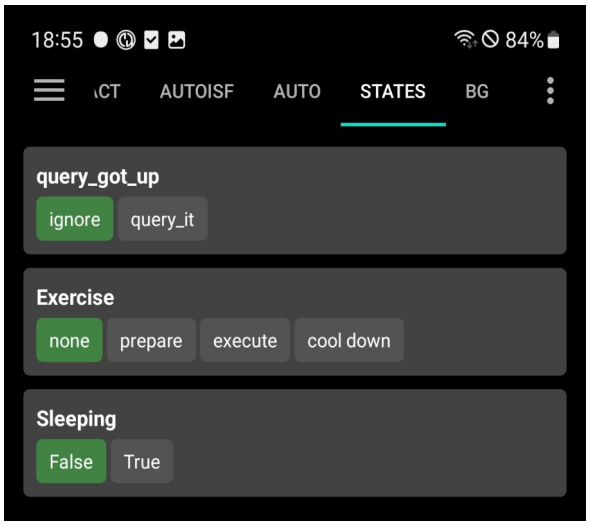




Status: 14/02/26 @ 15:37:20

Back to top

# The new Automation of States

This new feature is based on a PR proposal which was not merged into master AAPS. It was included in AutoISF since release 3.1.0 to test its usage. The idea is that a user can define any *state* and add several *values* to each *state*. For example he might define the state *Exercise* and define *values* for the different phases of exercise. In regular automation he can then check which *value* is currently active and take appropriate action like setting an exercise TT and switching the *state* to its next phase by picking the next *value*.



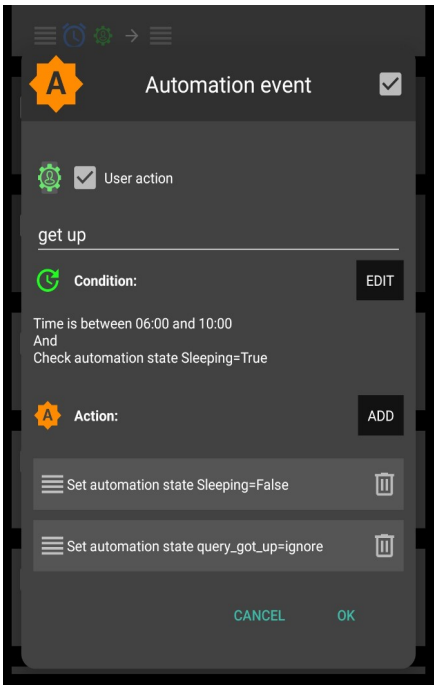

### Current prototype limitation

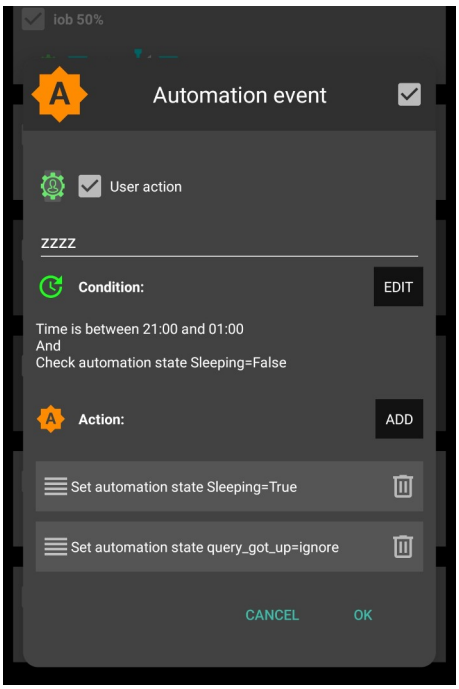
The user can click on the new *value* on this screen. The desired *value* is highlighted in green but in this prototype version the real assignment is not synchronised immediately and can take a while. Your safer bet is to click the *state* button to get to its edit screen. In there you can pick the new *value* – it might be shown already as green – and then exit via **OK** to really set the *value* immediately.

Also, displaying newly assigned values still takes significant time if assigned inside the AAPS code.

## *First real use example for disabling inactivity monitoring during sleep*

My sleep times are very irregular and I do not want to adapt those hours in the preferences every night. So I created a *state* Sleeping with its two obvious *values*. In addition I created two user action buttons which swap the values for going to bed and getting up, respectively. They trigger visibility in the morning or evening based on likely time windows and *state values*. The real trick now is to interact with those *states* inside the AAPS code rather than just by automations. This offers more complex behaviour. The slight downside is that in this case you have to stick to the names of *state* and *values* shown here. This will take precedence over the sleep hours and its enabling switch. If you do not want this new feature just do not create the *state* Sleeping – or misspell/rename it for your own use.

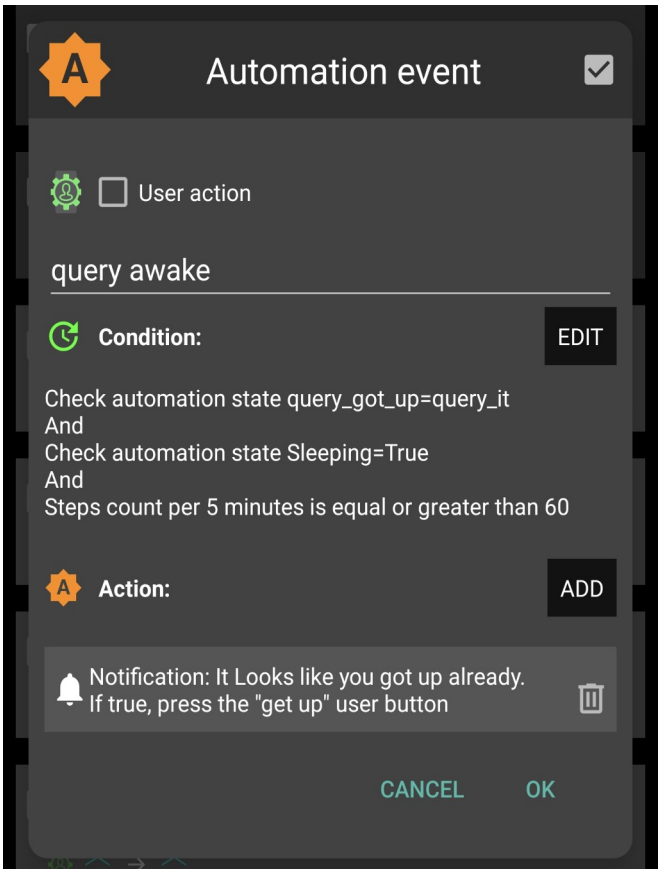



### Current prototype limitation

Disabling all automations in the configuration does not disable using these states inside the AAPS code.

How about that *state* query_got_up shown above? Its purpose is to remind the user to activate the get_up user button in case he forgot. Inside AAPS the end of the sleep window used in the fixed inactivity hours is checked as well and whether step counts look like activity beyond going the bathroom for a short sleep interruption. If these are satisfied then AAPS changes the value of *state* got_up from *ignore* to *query_it*. That triggers the automation *query awake* which creates a notification. The user can then swipe that notification off screen if he intends to go to bed again. The name of that automation and the text of that notification can be edited by the user. The names of the *state* got_up and its *values* are fixed.




The trigger level of which steps to check for finally getting up is still open and proposals are welcome. The current implementation checks if steps are detected younger than 15 minutes and additionally steps in the 15 -30 minute interval.

The user can add his personal step trigger level to the automation condition shown here. In the example here it is at least 60 steps in the last 5 minutes. But that depends on personal habits, flat layout, etc.

**Please note:** the standard step count trigger needed to be adapted for steps measured by the phone and under 1 minute loop conditions. It was also verified with 5 minute CGM. In our test setup for steps from the watch, neither the display of counts was possible nor did the step count trigger in the automations work (it is even the case in AAPS 3.3.2.0 master). If you experience problems, please let me know.

# Understanding messages

## *Loop power levels reported in AUTOISF tab*

Some formulations in version AutoISF3.0 used terminology like “Full loop disabled”. This was a left over from the test phase when their features were withdrawn but the messages stayed. These created a lot of confusion and were replaced. The table below shows the adapted messages, their impact and the condition when they apply.

| Message | Condition | What does it affect? |
|---|---|---|
| Loop allows maximum power | even target < 100 | increase in bg limited to 30%, otherwise no SMB;<br>actual SMB delivery ratio is max of fixed smb_delivery_ratio and linearly growing ratio |
| Loop allows medium power | even target >= 100 | increase in bg limited to 20%, the AAPS default, otherwise no SMB; actual SMB delivery ratio is either fixed smb_delivery_ratio or linearly growing ratio |
| Loop allows minimal power | odd target | no SMB, only TBR available for action |
| Loop power level temporarily capped | IOB > effective iobTH | Last SMB was capped to stay below iob threshold + 30% overrun; user defined iobTH potentially modulated by exercise mode, activity monitor and profile percent |
| Loop allows AAPS power level | no even/odd target option active | SMB enabled/disabled according to standard AAPS rules and settings;<br>no iobTH threshold is active |

## *AutoISF specific additions to the home screen*

The widget below contains all those additions as it is more or less identical to the top section of the home screen.


1. Time ago since last loop is shown in seconds as long as it is less than 100 seconds ago. Especially with Libre 1 minute values it was more or less frozen at 0m all day.

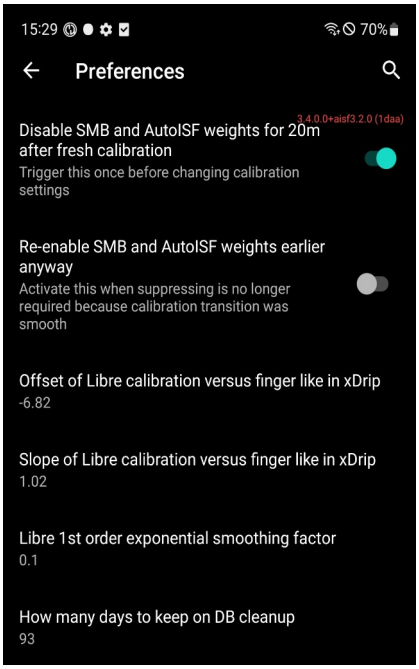
   Meanwhile this feature was added to master AAPS, too.

2. At the bottom right there are two percentages:

   **as:** denotes the Autosense value shown traditionally; *as* is only visible if it deviates from 100% just like in master AAPS

   **ai:** denotes the corresponding sensitivity change based on AutoISF; *ai* is always visible

# Using Libre 1 minute data in AutoISF

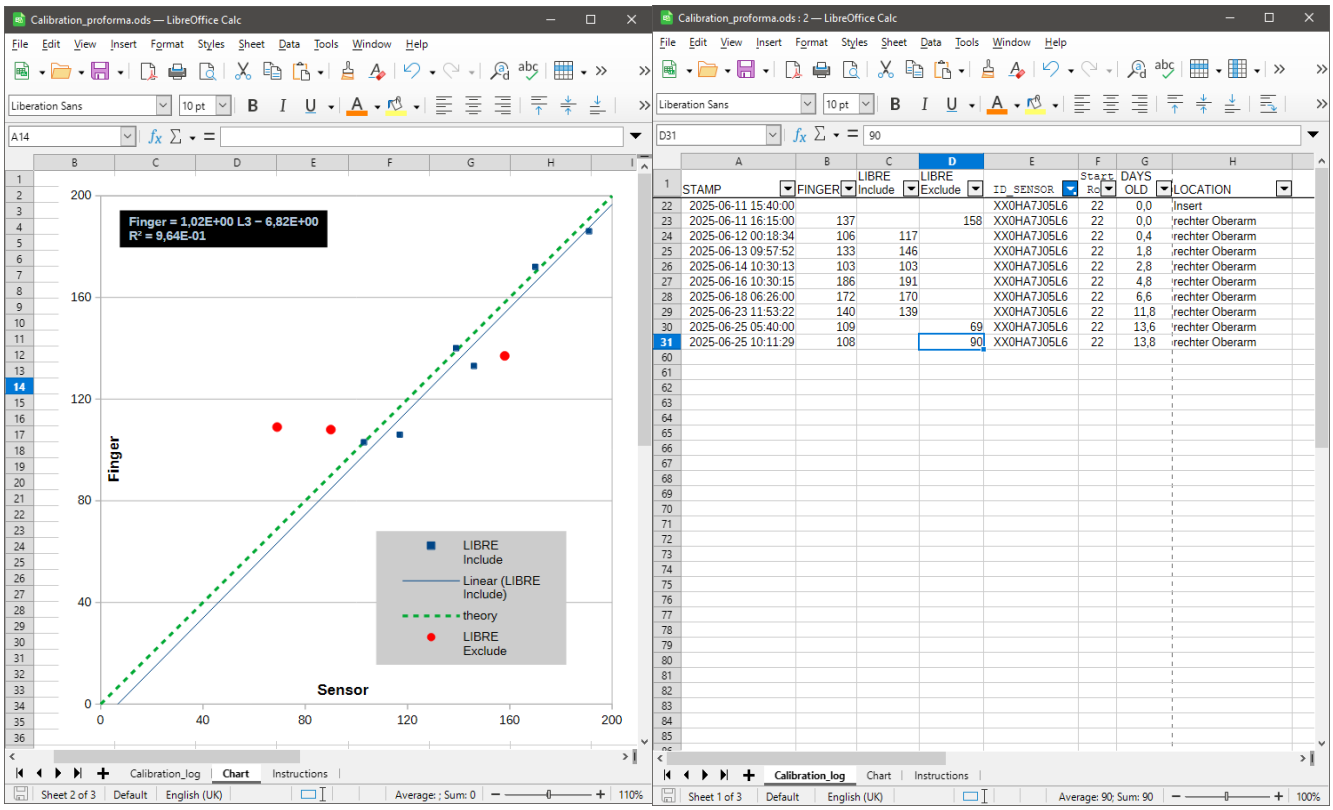



Consider as **background** that AAPS is configured in general to work with 5 minute data. However, as long as all calculation results like deltas are normalised to 5 minute intervals there is no numerical conflict. There were some bugs that needed to be fixed for enabling SMB every minute. Still, there are some other subjects to be aware of.

Every time a new Libre reading arrives, AAPS rebuilds an array of BG values 5 minutes apart. So it contains the Libre data from now, now-5m, now-10m, etc. This array is then treated just like coming from any other CGM. It can be smoothed, deltas can be calculated, etc. A minute later, i.e. at time now+1, this whole process starts over again and the new BG array then holds readings from now+1, now-4, now-9, etc. The readings in this new array are all different from the ones a minute earlier and only after 5 minutes the bulk of that array has the previous values in its history. One implication of this is an optical illusion[1] on the home screen. Like a serpent, the 5 minute subset (bright green dots) seems to creep along the white circles representing the Libre data. In AndroidAuto the deltas displayed do not correlate with the BG displayed now and at now-1m on top of being normalised to 5m anyway.

## *Libre calibration*

You can calibrate your Libre sensor directly in AAPS instead of xDrip. This means, if you are using Juggluco, there is no need to have xDrip in the data flow to AAPS. You enter values for slope and offset between finger prick values and sensor readings. The best way to get these values is to first enter them in




a spreadsheet and let it calculate the linear fit and show the parameter values. This could have been automated somewhat but, without methodical testing and experience, it is better to leave the decision to

1 For an example see Libre_serpent.mp4 (needs to be downloaded and animated locally)

Status: 14/02/26 @ 15:37:20

Back to top

the user which measurements to include in the fit. Maybe there is a drift in the sensors performance over time and you want to leave out older data. The example from LibreOffice[2] shown above includes such a drift. By filtering the current sensor can be selected. The older measurements were moved to the “exclude” column so they do not contribute to the fit. This way they are still visible in the chart. Also, readings from day 14 of that sensor were not really usable and also excluded. If some finger pricks were done in less stable conditions those can be excluded later when better data become available.

The values for slope and offset are entered in the Libre Specifics menu. The default values of slope=1.0 and offset=0.0 are defaults and disable calibration. The formula is

BgFinger = slope * BgSensor + offset

When testing a G7 sensor connected via Juggluco it only took minimal code adaptation such that G7 sensors can also be calibrated with the same method.

### *Libre smoothing*

The two standard smoothing options available in AAPS are not well suited for 1 minute data series. Therefore the 1st order exponential smoothing algorithm from Eversense/Esel was adapted and included directly in the AutoISF variant of AAPS. There is just one parameter which the user can use to strengthen or weaken the smoothing effect. This smoothing factor called alpha in the related literature can take values between 1.0 and 0.1 with the following effects:

1. A value of 1.0 is the default and does not smooth at all. It can be very useful to use for short periods when recalibrating the sensor. For details see the calibration section.
2. A value of 0.0 has maximum effect and results in a perfectly flat and constant line. Surely this is not what we want either and therefore the allowed range is limited by 0.1 but not less.
3. A typical value of 0.1 for a noisy Libre provides significant smoothing but delays the signal by about 10 minutes. Values lower than 0.1 for alpha are therefore not recommended. As an offset for that delay the parabola fit can now be executed with data covering just 10 minutes instead of 20 minutes of raw data in the previous version of AutoISF. This recovers most of the delay and testing showed that recognition of acceleration is not delayed compared to the previous version. The smoothing improves fit correlation to be close to 1.0 most of the time. Therefore no “penalty” gets deducted and FCL with AutoISF still works.


The screenshot shows a direct comparison between raw Juggluco data and calibrated, smoothed data in AAPS.

Using this strong smoothing is theoretically available for users of DynamicISF but is not recommended. That algorithm relies heavily on absolute BG values. Especially maximum and minimum values are significantly delayed with this smoothing algorithm and reach those levels only asymptotically. If the trend changes then the extreme values will even be capped by this smoothing.

G7 sensors can also be smoothed with the same method but most likely with weaker smoothing factor alpha like 0.5.

2 See https://github.com/ga-zelle/autoISF/tree/A3.4.0.0_ai3.2.0/Calibration_proforma.ods

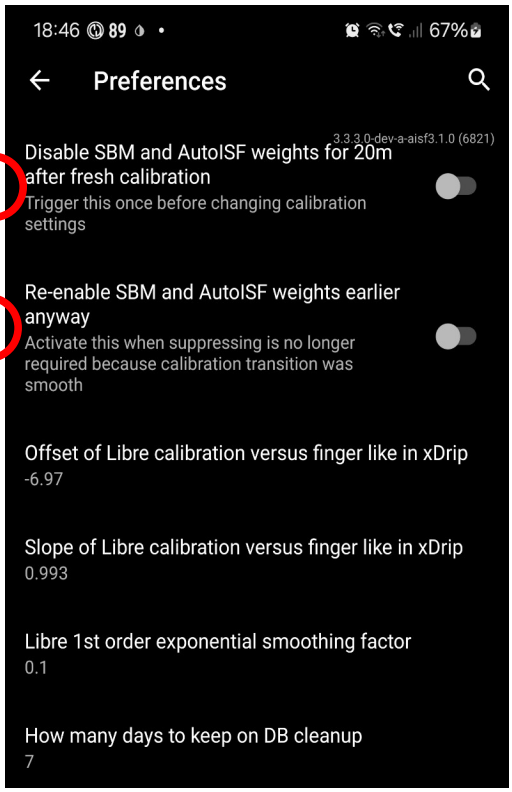
Why not use **xDrip**? It can be used as an alternative to include smoothing or send data only every 5 minutes. That remains an option if users prefer it or have other reasons to include xDrip in the process chain like Libre calibration. But be aware of a trap in xDrip: when a Libre reading arrives earlier than 60 sec that value is rejected by xDrip and no loop is triggered. When you need xDrip for alarms etc. you better have Juggluco send data to AAPS and xDrip in parallel to avoid such drop outs. When AutoISF detects readings coming from xDrip then it will not apply any calibration or smoothing as described above.

One other important aspect when using 1 minute readings is related to the ***smb_delivery_ratio***. Normally we want large SMB as soon as possible to be advantageous. Here, the opposite proved to be better. Using several small steps rather than a few large steps resulted in a much smoother ride. So AutoISF now allows ratios down to 0.1 where 0.2 seems to be a good starting value. The added advantage of such lower ratios is that responses to sensor noise are much smaller and therefore less critical. The more frequent activation of the pump reminds me of the B30 method favoured in AIMI.

One last aspect of using 1 minute data is the **drain on battery**. In my case it mainly impacts the Combo battery because of more frequent TBR adjustments and more but minor SMBs. Its battery now lasts only 3 weeks compared to 4 weeks when using the 5 minute readings from Eversense. Similarly, the battery of the watch can be used much more. One other trick to reduce phone battery usage is related to the size of the database. It holds 5 times the data for BG, temp basals, etc. The option to clean-up database in the maintenance section can now clean-up down to 7 days instead of the default full quarter. That minimum period can be set in the Libre specific settings menu.

**In summary** the recommended data flow is very simple: Juggluco → AAPS → calibration & 1$^{st}$ order exponential smoothing. Set an SMB delivery ratio below 0.5.

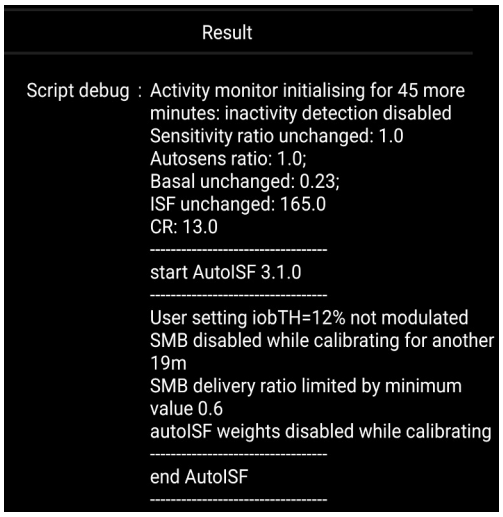
## Smooth transition when calibrating




When calibrating there will most likely be a step change in Bg values. This step on its own is not caused by a real and sudden change in blood glucose. Therefore actions need to be taken to avoid false boluses for typically 20 minutes until the parabola fit is based on data after the calibration step:

1. Disable AutoISF weights;
2. Disable SMBs;
3. Disable smoothing for one loop event after the calibration takes effect. This will restart the Bg curve at the new level immediately and then resume smoothing;
4. After 20 minutes re-enable all above listed settings if they were active before.

Clearly managing all this manually is prone to human error. Therefore 2 switches were added to the Libre special menu right before the calibration parameters in order to automate this as much as possible:




1. Click this before editing the calibration settings. It will start the 4 actions listed above. There is no need to change TT or any other setting like you would do manually. Everything related will just be ignored internally until the 20 minutes have expired. The AUTOISF tab will show how many minutes are left before things become active again like they would be without the calibration.

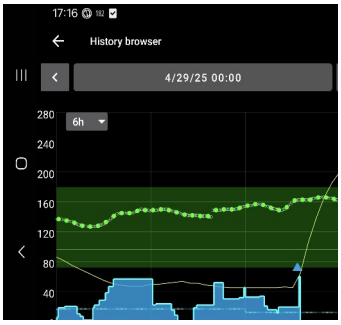
   Please note that option (1) will be switched off again in the menu display when receiving the next Bg reading. For checking the current state consult the AUTOISF tab.

   Please also note that to be on the safe side the smoothing will only resume after 2 minutes just in case entering the calibration parameters takes a while.

2. Click this if you feel that the Bg step was negligible and this whole transition process could already end now.

   Please note that option (2) will remain as true and be disabled automatically when option (1) gets triggered again.

   Please also note that de-activating this option yourself early will resume the restrictions until the 20 minutes have expired.




Here is an example result of such a small step change without SMB. Compare against the SMB for a similar change about 60 minutes later. Please note this can also be useful in other cases of unphysiological steps in BG like a new sensor was started.

Status: 14/02/26 @ 15:37:20

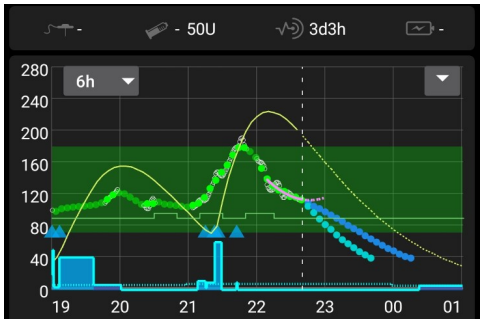
Back to top

# Show key interim values of AutoISF

What is this about? Before the 3.2 AutoISF version, you needed the help of the emulator to get graphical information about those past values. Now you can see them directly on the AutoISF home screen and in the History Browser.

How you arrange the information in the small graphs and deciding what to show or not is a matter of personal style and preferences. It is easy to try various layouts. For instance, I ended up with a subset of the AutoISF factors at the stop and iob_TH just beneath as my key information.

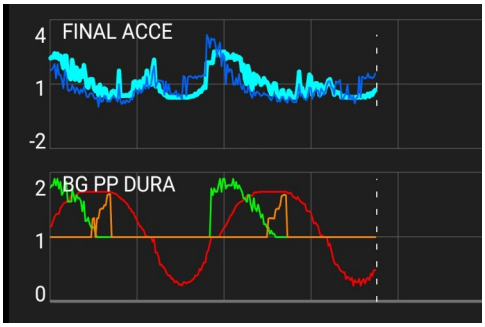
## *Showing the fit parabola*




It is an overlay in the BG main graph. The length of history is given by the fit duration, i.e. a minimum of 15m (or 10m for Libre, resp.) up to a maximum of 45m. Also, an extrapolation for the next 20 minutes is shown as a series of more spread out dots. This enables to see the near future at a quick glance and therefore the display of *acc:* at the top of the home screen was discontinued. When you observe that extrapolation regularly while approaching a maximum or minimum you can learn how trustworthy that curve is for yourself. That helps keeping your calm and not overreact next time.

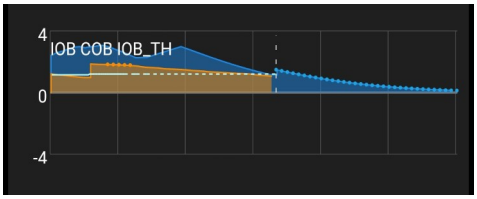
What I had not expected was the widening of the parabola with most new loops. The shorter intervals with Libre let you see it nearly live. But this widening is the visual sign of reduced acceleration. Initially I was caught by surprise. But this is exactly the aim of the algorithm, namely reduce acceleration to get into quieter waters.

## *Showing the AutoISF factors*




These can be shown as line charts in the small graphs. Within each graph frame the factors are all to the same scale. To add other values makes no sense anyway as they bring their own scale or might even disturb the required unique scaling. To distinguish larger values for acce and final from the other, smaller ones they can be assigned to different graph frames.

## *Showing iobTH*




This can be shown as dotted line. To be exact the effective iobTH is shown as modulated by exercise mode or similar. It makes sense to add IOB and COB to the same small graph to get common scaling between them.

Status: 14/02/26 @ 15:37:20

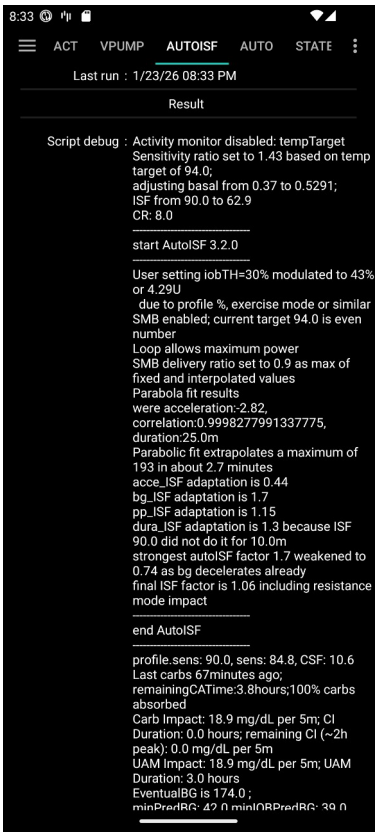
Back to top

### *Show debug script in AAPSClient*

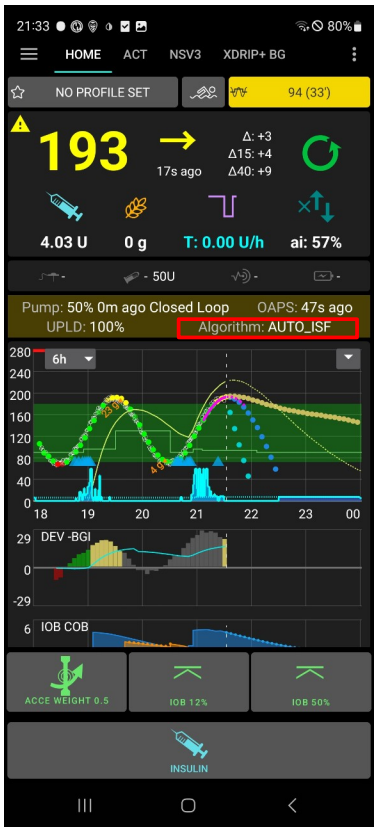
Apart from the fitted parabola none of the above new graphics can be shown in AAPSClient because the related data are not sent to Nightscout. Therefore they cannot be reloaded to the client from Nightscout. By the way, the same holds for HR and STEPS.

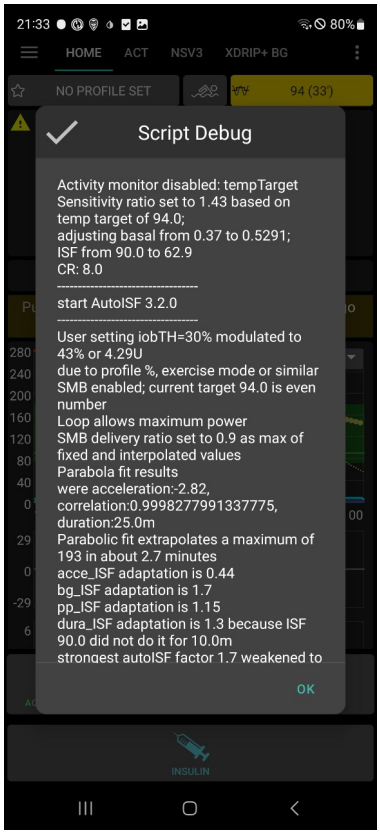
Nevertheless and as an offset, you can now at least request in the client to show the Script Debug from the master because it is so important for AutoISF. For that purpose the field Algorithm was added besides e.g. Pump and OAPS. It displays the plugin and algorithm active in master. Upon clicking it displays the active Script Debug. Here is an example:

MASTER


CLIENT




Status: 14/02/26 @ 15:37:20

Back to top

## **Attachment 1**: Table of all AutoISF settings for Hybrid Closed Loop mode[3]

| Name | Use | Min – Max | Default | Useful initial value for adults[4] |
|---|---|---|---|---|
| **Settings related to the 4 glucose ISF weights** | | | | |
| enable_autoISF | Use any of the 4 glucose weights for scaling the glucose ISF factors | False - True | False | |
| autoISF_min | Lowest ISF factor allowed | 0.3 – 1.0 | 1 | 0.7<br>like autosense_min |
| autoISF_max | Highest ISF factor allowed | 1.0 – 3.0 | 1 | 1.2<br>like autosense_max |
| **Settings related to acce_ISF, i.e. related to glucose acceleration** | | | | |
| bgAccel_ISF_weight | Strength of acce_ISF contribution with positive acceleration | 0.0 – 1.0 | 0 | 0.02 |
| bgBrake_ISF_weight | Strength of acce_ISF contribution with negative acceleration | 0.0 – 1.0 | 0 | 0.01 |
| **Settings related to pp_ISF, i.e. glucose delta** | | | | |
| pp_ISF_weight | Strength of effect | 0.0–0.15 | 0 | 0.005 |
| **Settings related to bg_ISF, i.e. glucose deviating from target** | | | | |
| lower_ISFrange_weight | Strength of bg_ISF effect when bg<target | 0.0 – 2.0 | 0 | 0.2 |
| higher_ISFrange_weight | Strength of bg_ISF effect when bg>target | 0.0 – 2.0 | 0 | 0.2 |
| **Settings related to dura_ISF, i.e. related to glucose “frozen” at high levels** | | | | |
| dura_ISF_weight | Strength of dura_ISF effect | 0.0 – 3.0 | 0 | 0.2 |
| **Settings related to SMB delivery size** | | | | |
| smb_delivery_ratio | Increase the AAPS standard 0.5 of Insulin required | 0.1 – 1.0 | 0.5 | 0.6 |
| smb_delivery_ratio_min | Minimum for linearly rising ratio at and below target_bg | 0.1 – 1.0 | 0.5 | 0.5 |
| smb_delivery_ratio_max | Maximum for linearly rising ratio at and above target_bg + smb_delivery_ratio_bg_range | 0.5 – 1.0 | 0.9 | 0.8 |
| smb_delivery_ratio_bg_range | Width of bg range to reach maximum ratio | 0.0 – 100 | 0 | 40 |
| smb_max_range_extension | Multiplier for maxSMBBasalMinutes and maxUAMSMBBasalMinutes | 1.0 – 5.0 | 1 | 1.5 |
| **Settings related to enabling/disabling SMB by even or odd target values** | | | | |
| enableSMB_EvenOn_OddOff_always | Enable SMB depending on profile target value type | False - True | False | |
| **Settings related to IOB threshold** | | | | |
| iob_threshold_percent | Very important safety parameter in FCL mode; caps or disables SMB when exceeded | 10-100 | 100 | 60 |
| **Settings related to scaling the activity monitor impact** | | | | |
| activity_scale_factor | Strength of effect | 0.0 – 1.5 | 1.0 | |
| inactivity_scale_factor | Strength of effect | 0.0 – 1.5 | 1.0 | |

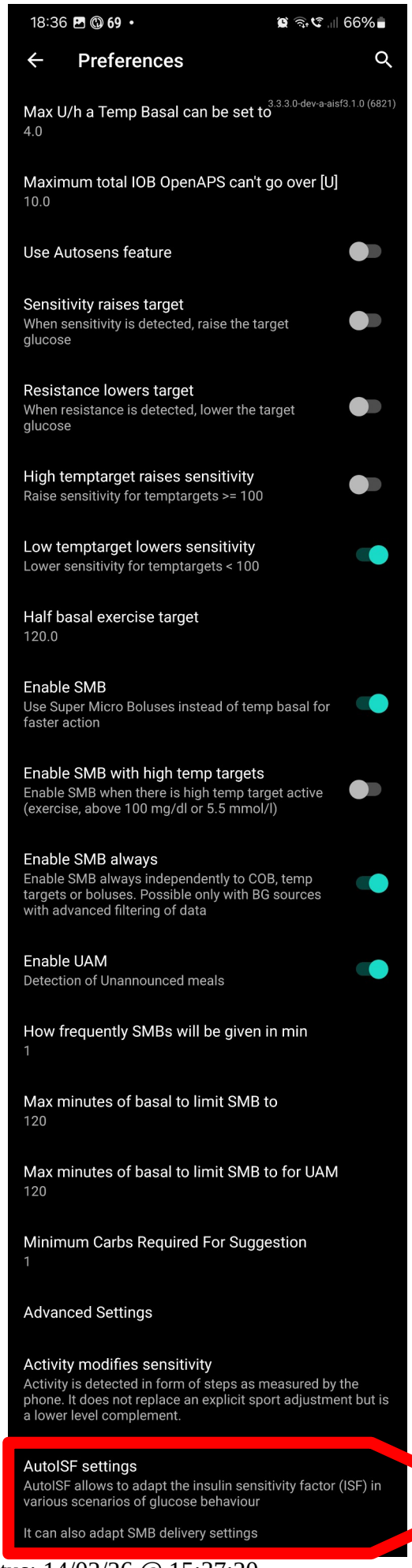
3 For **F**(ull)**C**(losed)**L**(oop) mode values differ and you must follow the FCL e-Book guidelines
4 For children there is minimal experience and you need to use much softer settings

# Attachment 2: Table of Libre specific settings in AutoISF

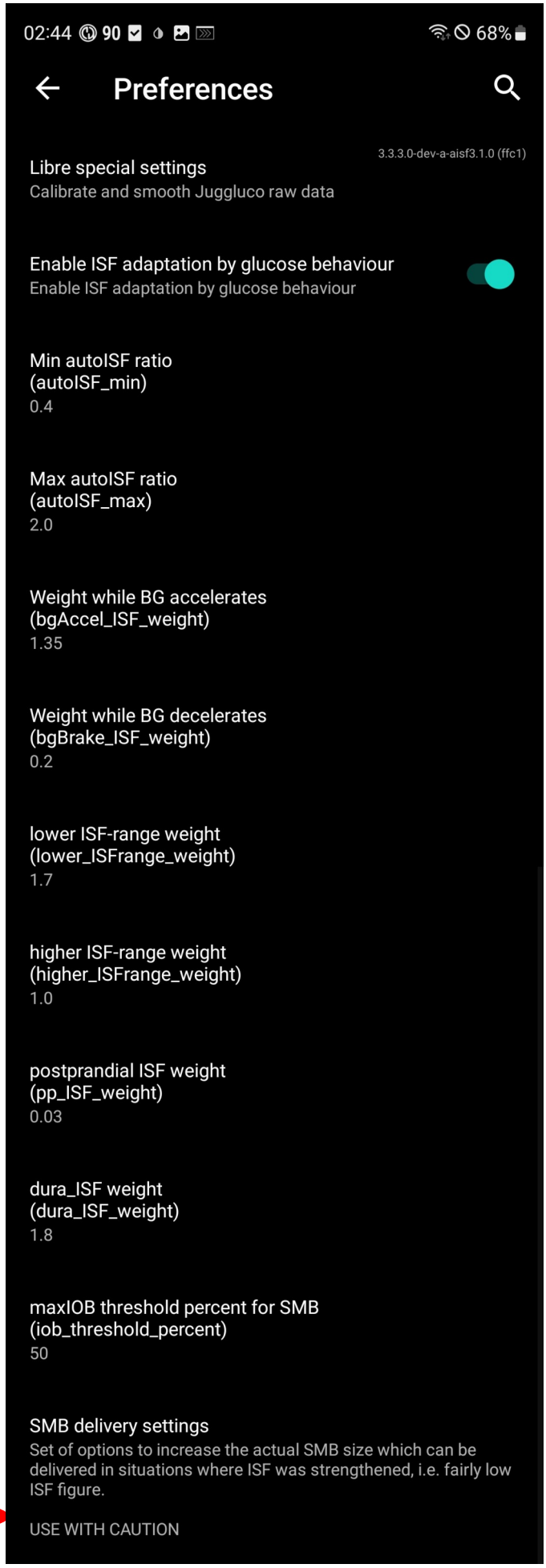
| Name | Use | Min – Max | Default | Useful initial value |
|---|---|---|---|---|
| **Settings related to calibrating raw readings imported from Juggluco** | | | | |
| Disable SMB and BG related weights after new calibration | Use it to avoid reactions due to new BG offset | False - True | False | False |
| Re-enable loop reactions anyway | Use when new calibration transition looks smooth | False - True | False | False |
| offset | calibrated = slope*raw + **offset** | -50 - 50 | 0.0 | Your average across previous sensors |
| slope | calibrated = **slope***raw + offset | 0.5 – 1.5 | 1.0 | Your average across previous sensors |
| **Settings related to smoothing calibrated readings from above** | | | | |
| alpha | Strength of effect:<br>1.0 – no smoothing<br>0.1 – maximum smoothing | 0.1 – 1.0 | 1.0 | 0.1 for noisy sensors<br>0.5 for fairly smooth sensors like G7 |
| **Settings related to DB size and battery consumption** | | | | |
| days to keep | Maintenance – Database Clean-up | 7 – 93 | 93 | 14 |

## Attachment 3: Screenshot of AutoISF menu

AUTOISF Plugin




AutoISF Menu




Status: 14/02/26 @ 15:37:20

Back to top